COSC 3430 · ARTIFICIAL INTELLIGENCE

End-to-End Machine Learning Project

Part 4: explore and visualize the training data - correlations

ROADMAP

Ten steps, over several code-along lectures

1 Look at the big picture
2 Get the data
3 Explore and visualize the data
4 Create the test data
5 Prepare the data for learning
6 Select a model and train it
7 Fine-tune the model
8 Present your solution
9 Launch the production system
10 Monitor and maintain it

Today: back to step 3, a second and deeper pass - on the training set only.

See also: Géron's ML project checklist (github.com/ageron/handson-mlp)

RECAP AND BRIDGE

Where part 3 left off

1. The data was split into strat_train_set and strat_test_set, stratified on median_income categories
2. The test set is sealed away - no data snooping - until the model is ready to be evaluated
3. Today we work on a fresh copy of the training set only, called housing

→ SWITCH TO EDITOR · RECAP CODE

One block gets us back to where we left off

```
housing_full = pd.read_csv(url)

housing_full["income_cat"] = pd.cut(
    housing_full["median_income"],
    bins=[0, 1.5, 3.0, 4.5, 6.0, np.inf],
    labels=[1, 2, 3, 4, 5])

strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2,
    stratify=housing_full["income_cat"],
    random_state=42)

for df in (strat_train_set, strat_test_set):
    df.drop("income_cat", axis=1, inplace=True)

housing = strat_train_set.copy()
```

Same four moves as part 3, back to back: read, cut into income categories, split with the same random_state=42, drop the helper column.

```
16512 rows, 10 columns
total_bedrooms: 16344 non-null
```

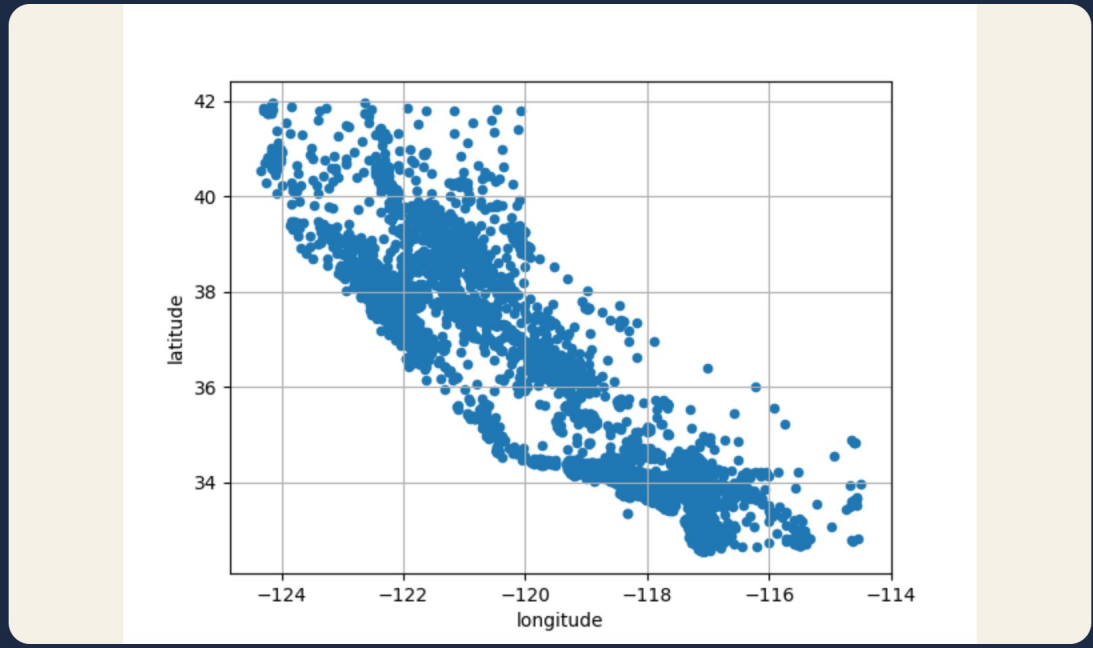
```
print(housing.info())
```

→ SWITCH TO EDITOR · CODE-ALONG 1

Create a map

We have geographical information for every district - plot it directly as a scatterplot.

`kind="scatter", x="longitude", y="latitude"`




```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
```

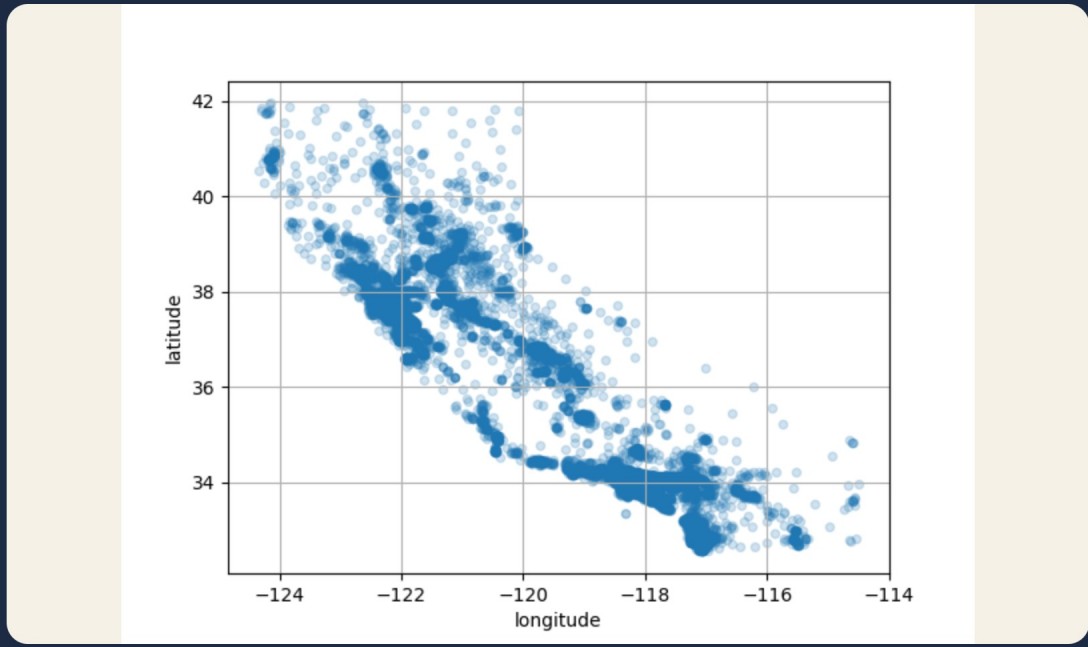
→ SWITCH TO EDITOR · CODE-ALONG 2

Making a messy map readable: alpha

The parameter is hidden in ****kwargs** - check help(plt.scatter), not the pandas docstring.

`alpha=0.2`

0 = transparent, 1 = opaque. Where many points overlap, the area appears darker.




```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True, alpha=0.2)
```

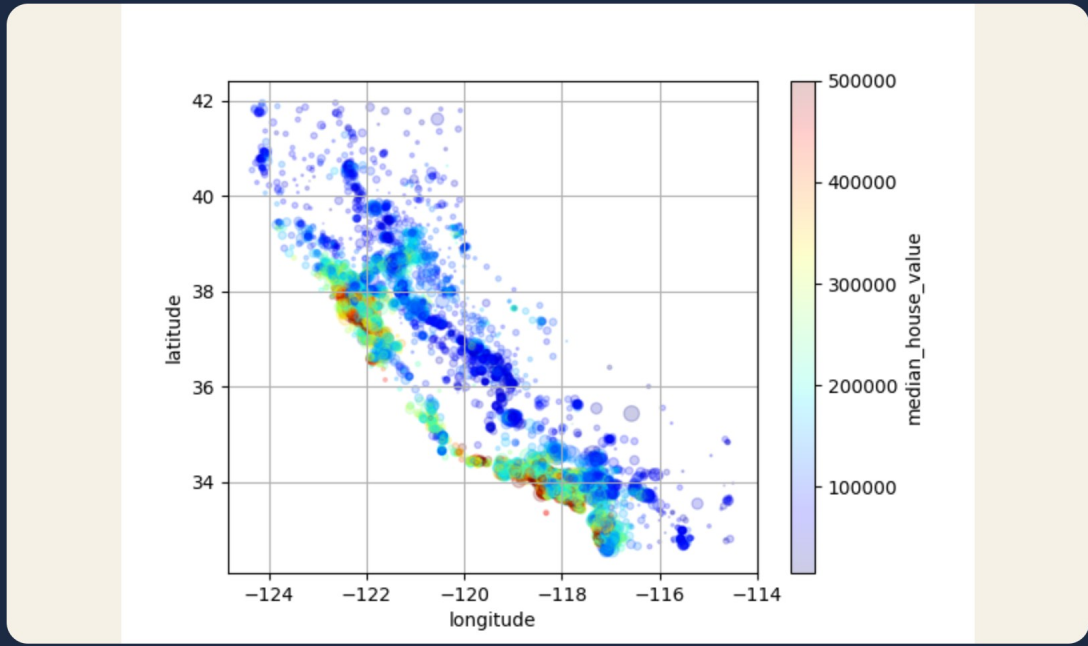
→ SWITCH TO EDITOR · CODE-ALONG 3

Adding population and median_house_value

Two more variables turn the 2D plot into an effective 3D plot: dot size and dot color.

```
s=population/100  (dot size)
```

```
c=median_house_value, cmap="jet"  (dot color)
```




```
s=housing["population"]/100, c="median_house_value", cmap="jet", colorbar=True
```

CORRELATION COEFFICIENTS

Pearson's coefficient, via corr()

A pandas method that measures how two numeric columns grow and fall together - between every pair of numerical attributes at once.

$$r = \frac{\Sigma\,(a - \bar{a})(b - \bar{b})}{\sqrt{\,\Sigma(a - \bar{a})^2 \cdot \Sigma(b - \bar{b})^2\,}}$$

Cosine similarity in disguise: subtract the means, and r = a·b / (‖a‖·‖b‖) = cos θ of the two centered column vectors (lecture 4).

Four facts

Ranges from -1 to 1 · the matrix is symmetric · the diagonal is always 1.0 · it can accept a custom method, not just Pearson's

A toy example first

Two columns, dogs and cats, four rows - two small histograms, easy to compute by hand before we run corr() on the real data

covariance of a and b, divided by (std-dev of a · std-dev of b)

→ SWITCH TO EDITOR · CODE-ALONG 4

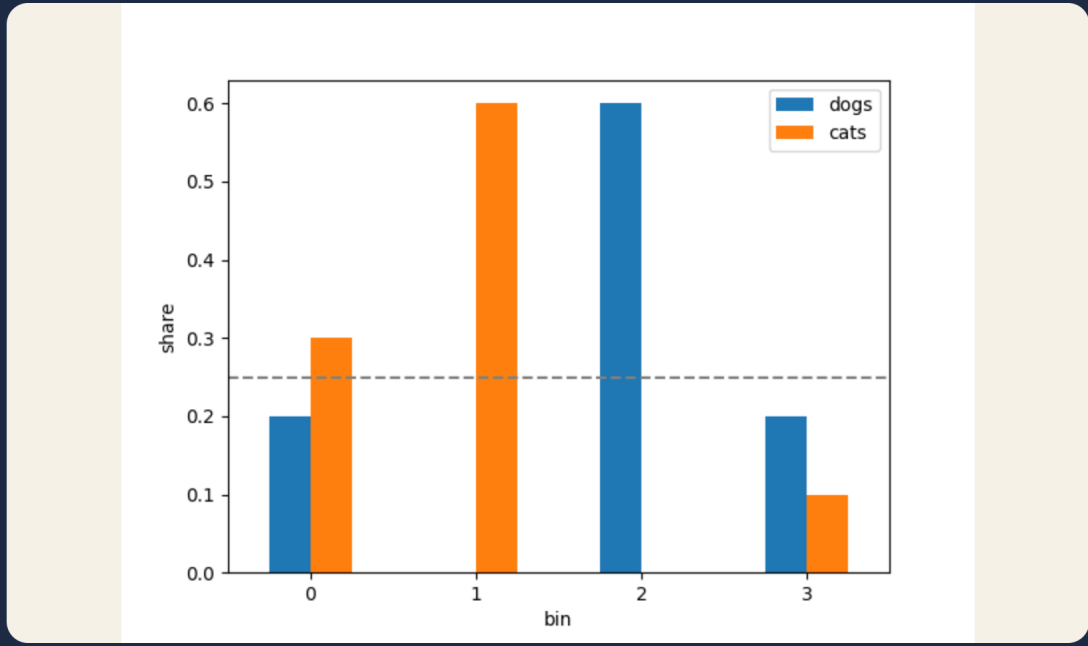
The toy data as a bar chart

Each column already is a histogram: one share per bin. So plot the values as bars - hist() would count how often each value occurs instead.

`kind="bar"`

`plt.axhline(0.25, linestyle="--")`

Dashed line: the mean of both columns, 0.25. In each bin, do the two bars sit on the same side of it, or on opposite sides?




```
df.plot(kind="bar", rot=0, xlabel="bin", ylabel="share")
```

→ SWITCH TO EDITOR · CODE-ALONG 5

A toy correlation, by hand

Two histograms (each column adds up to 1.0) · both means = 0.25 · a = dogs − ā, b = cats − b̄

	dogs	cats	a	b	a·b	a²	b²
0	0.2	0.3	-0.05	+0.05	-0.0025	0.0025	0.0025
1	0.0	0.6	-0.25	+0.35	-0.0875	0.0625	0.1225
2	0.6	0.0	+0.35	-0.25	-0.0875	0.1225	0.0625
3	0.2	0.1	-0.05	-0.15	+0.0075	0.0025	0.0225
Σ					-0.17	0.19	0.21

Rows 1 and 2 are mirror images - they supply almost the whole numerator (-0.175 of -0.17).

Plug into the formula

$$r = -0.17 \,/\, \sqrt{(0.19 \cdot 0.21)}$$
$$= -0.17 \,/\, 0.19975$$
$$= \mathbf{-0.851}$$

Same as df.corr(): dogs and cats move strongly in opposite directions.

> **Sign of a·b:** negative where the two sit on opposite sides of their mean (rows 0–2), positive on the same side (row 3).
>
> **The denominator** only rescales: r lands between −1 and 1, in any units.

```
print(df.corr()) # method="pearson" is the default
```

→ SWITCH TO EDITOR · CODE-ALONG 6

A custom method: how much do they overlap?

corr(method=...) also accepts your own function: two columns in, one number out. This one, from the corr() docstring, adds up the smaller value in each row.

```
def histogram_intersection(a,b):
    v=np.minimum(a,b).sum().round(decimals=1)
    return v
```

```
min(0.2, 0.3) = 0.2    min(0.0, 0.6) = 0.0
min(0.6, 0.0) = 0.0    min(0.2, 0.1) = 0.1
sum = 0.3
```

Same data, two questions

method	dogs/cats	asks
pearson	-0.851	Do they grow and fall together?
histogram_intersection	0.3	How much do the two histograms overlap?

Rows 1 and 2 add nothing to the overlap (one value is 0). A sum of non-negative minimums can never be negative - unlike Pearson's r.

```
print(df.corr(method=histogram_intersection))
```

RESULT

What correlates with median_house_value?

Feature	r
median_house_value	1.000
median_income	0.688
total_rooms	0.137
housing_median_age	0.102
households	0.071
total_bedrooms	0.055
population	-0.020
longitude	-0.051
latitude	-0.140

Reading it

House value goes up when income goes up. Prices go slightly down the further North you go.

median_income wins

By far the strongest linear relationship - the same feature used for stratified sampling in part 3.

```
corr_matrix["median_house_value"].sort_values(ascending=False)
```

QUESTION

A coefficient close to zero means "no linear correlation." What does that mean – and what does it *not* mean?

ANSWER

No linear relationship, not no relationship

What it means

Fitting a line through the scatterplot of two uncorrelated variables would give a flat line. Changes in x give you no predictive power over y.

What it does *not* mean

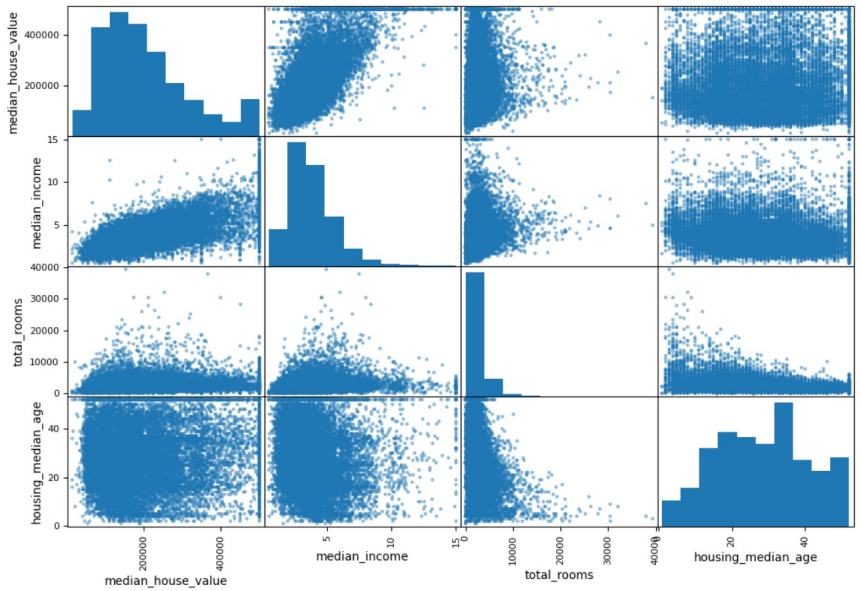
There is no linear relationship - but there could still be a strong *non-linear* curve describing how x and y grow together.

→ SWITCH TO EDITOR · CODE-ALONG 7

Correlation, visually: scatter_matrix()

9 features against 9 features would be 81 panels. Focus on the four most promising, chosen from the correlation table.

```
median_house_value, median_income,
total_rooms, housing_median_age
```




```
scatter_matrix(housing[attributes], figsize=(12,8))
```

ANSWER

What do you learn from the matrix?

The diagonal

Populated with a histogram of each attribute by default - you can pick a different plot for it, as we'll see next.

One clear winner

All three attributes look promising, but the panel with the clearest point trend - greatest predictive potential - is median_income.

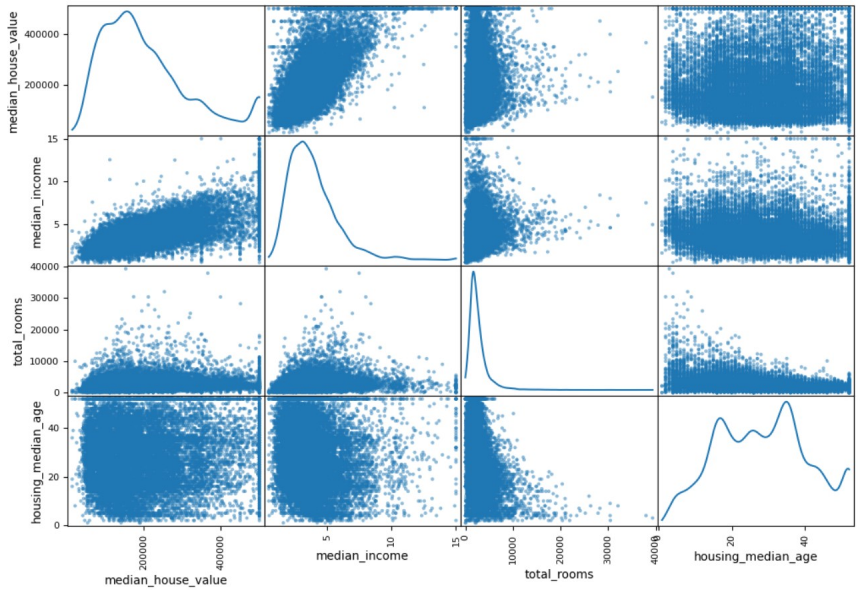
→ SWITCH TO EDITOR · CODE-ALONG 8

The diagonal as a density plot

One parameter swaps the histogram diagonal for a smoothed density curve.

`diagonal='kde'`

Noticeably slower to compute - the insight gained over the histogram isn't overwhelming.



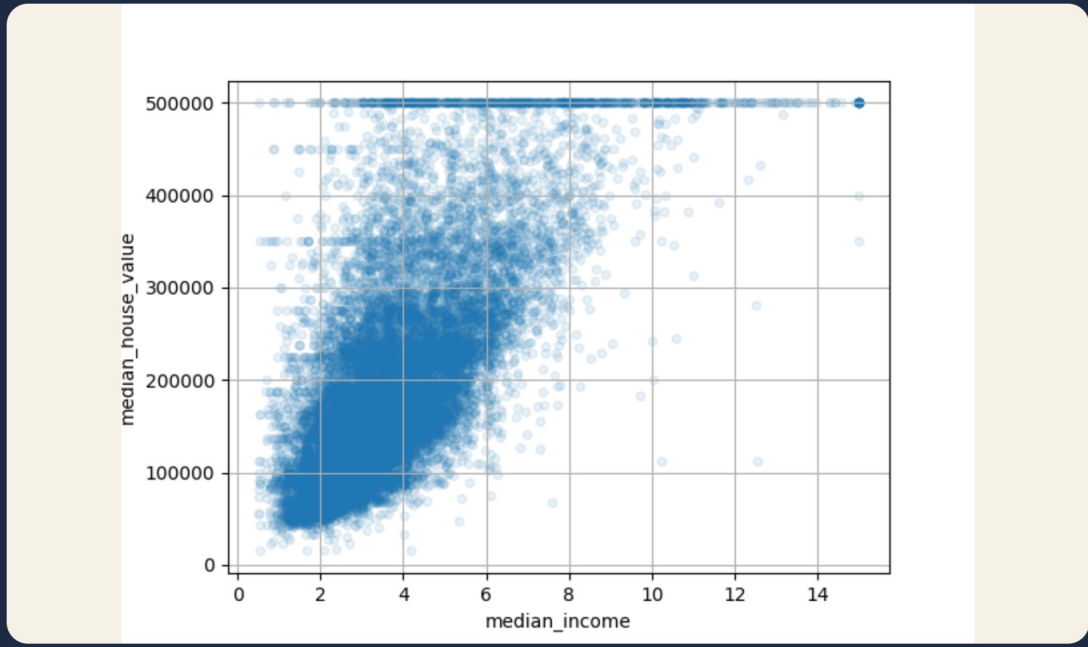

```
scatter_matrix(housing[attributes], figsize=(12,8), diagonal='kde')
```

→ SWITCH TO EDITOR · CODE-ALONG 9

Zoom in on the most promising panel

A dedicated scatterplot of the single strongest relationship: income against house value.

```
x="median_income", y="median_house_value", alpha=0.1
```




```
housing.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1, grid=True)
```

ANSWER

A strong trend - and a few artifacts

The trend

Strong and clearly visible, despite the noise. The $500,000 price cap shows up as a flat line at the top.

Suspicious lines

Straight horizontal lines near $450,000 and $350,000 too - likely collection artifacts, not real patterns.

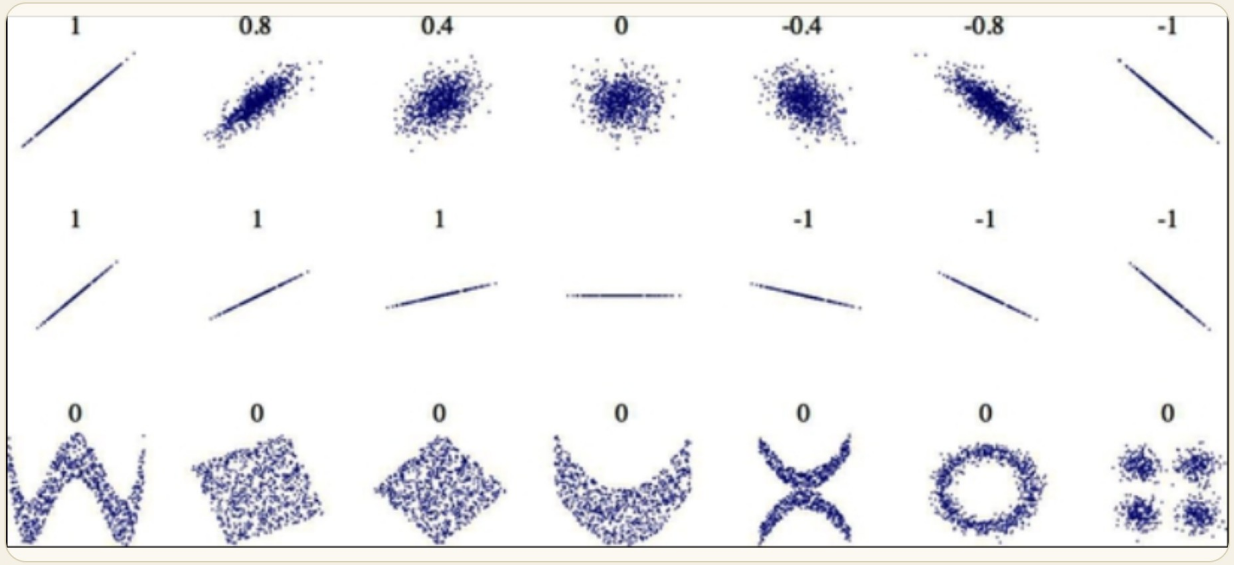
Why it matters

Train on these as-is and the model may learn "income X plateaus at $450,000" - worth removing those districts.

Can you be *sure* a regular pattern like this is an artifact? No - nature rarely produces perfect patterns, so it's a strong inference, not a proof.

REFERENCE

Standard correlation coefficients, at a glance



Bottom row: r = 0

All nonlinear relationships - a reminder that zero does not mean unrelated.

Second row: slope ≠ coefficient

Unit conversions (inches, nanometers) have wildly different slopes but the same r = 1 - the measure is normalized by standard deviations, so scaling cancels out.

→ SWITCH TO EDITOR · CODE-ALONG 10

Experimenting with attribute combinations

The raw totals mostly measure how big a district is. Combine them into features that describe a typical house instead:

```
rooms_per_house = total_rooms / households
```

```
bedrooms_ratio = total_bedrooms / total_rooms
```

```
people_per_house = population / households
```

This is what exploring is for: it gives you ideas for new features, and the correlation checks them before you build a model.

Feature	r
median_income	0.688
rooms_per_house (new)	0.144
total_rooms	0.137
total_bedrooms	0.055
people_per_house (new)	-0.038
bedrooms_ratio (new)	-0.256

bedrooms_ratio is now second only to income - much stronger than the bedroom and room totals it is built from.

```
corr_matrix["median_house_value"].sort_values(ascending=False)
```

WRAP-UP

What the training set told us

- Map plots with alpha, size and color reveal density and price by location
- corr() gives Pearson's coefficient for every pair of numeric columns: -1 to 1, symmetric, diagonal = 1.0
- Close to zero means no *linear* relationship - not no relationship
- median_income is the strongest predictor of median_house_value ($r \approx 0.69$) - the same feature used for stratified sampling
- scatter_matrix() confirms it visually; the zoomed scatterplot shows the trend and some likely data artifacts
- Attribute combinations: bedrooms_ratio ($r \approx -0.26$) beats the raw totals it is built from

Next: prepare the data for learning - cleaning, encoding, scaling.